

CryptoTracker

Crypto-Currency Portfolio Analyzer

Comprehensive Project Documentation

Semester 3 – Sprint 1 | Individual Project

Submitted By:

Kishan Ojha

Roll Number / Student ID:

15009674160

Course / Subjects:

- Data Structures & Algorithms II
 - NoSQL Databases – MongoDB & Firebase
 - Business 2 – Customer Acquisition, Sales & Operations
-

CryptoTracker - Comprehensive Project Documentation

1. Project Title and Overview

Project Title: CryptoTracker - Crypto Portfolio Management App

Project Summary / One-line Description:

A comprehensive cryptocurrency portfolio management application that allows users to track, analyze, and optimize their crypto investments with real-time data and advanced analytics.

Purpose & Problem Solved:

CryptoTracker addresses the challenge of managing and monitoring cryptocurrency investments across multiple platforms. It provides a unified dashboard for tracking portfolio performance, analyzing asset allocation, and gaining insights through advanced analytics. The platform helps both novice and experienced investors make informed decisions by offering real-time data visualization, performance metrics, and predictive insights.

2. Theme Alignment

This project fits into the theme "**Customer Acquisition to Loyalty: Sales Funnel & Operations Dashboard**" by:

1. **Lead Capture & Qualification:** Signup/login flows with multiple authentication options (email/password, Google, crypto wallet).
2. **Engagement & Retention:** Dashboard features, portfolio tracking, and analytics keep users engaged with their investments.
3. **Value Delivery:** Advanced insights, alerts, and performance metrics provide ongoing value to encourage loyalty.

4. **Data-Driven Operations:** User portfolio analysis dashboard enables understanding of customer behavior and platform usage patterns.
-

3. Objectives

Technical Objectives

- Real-time cryptocurrency portfolio tracking with Firebase integration
- Responsive UI with dark/light theme support using Tailwind CSS
- Authentication system supporting multiple login methods
- Data visualization components for portfolio analytics
- Wallet authentication using cryptographic signature verification

Business Objectives

- Facilitate customer acquisition through intuitive onboarding and multiple authentication options
- Drive engagement through portfolio analytics and insights
- Encourage loyalty with personalized recommendations and alerts

- Enable data-driven decision making through user behavior analysis dashboard
-

4. Technology Stack

Frontend

- Next.js 13+ (App Router)
- React 18+
- TypeScript
- Tailwind CSS v4
- Recharts (data visualization)
- Framer Motion (animations)

Backend

- Next.js API Routes
- Firebase Authentication
- Firebase Firestore
- Firebase Admin SDK

Libraries & APIs

- ethers.js & viem (wallet authentication)

- Reown AppKit (wallet connection)
- CoinMarketCap API (crypto data)
- Google Authentication provider

Development Tools

- ESLint
 - npm / yarn
-

5. System Architecture

graph TB

```
A[User Interface] --> B[Next.js Frontend]
B --> C[Firebase Authentication]
B --> D[Next.js API Routes]
D --> E[Firebase Firestore]
D --> F[CoinMarketCap API]
E --> G[Firebase Admin SDK]
B --> H[Recharts Visualization]
B --> I[Wallet Connection]
I --> J[Ethers.js/Viem]
```

Data Flow

1. User interacts with frontend UI
2. Authentication requests go to Firebase Authentication
3. Portfolio/user data stored/retrieved from Firebase Firestore

4. Cryptocurrency market data fetched from CoinMarketCap API
 5. Wallet authentication uses cryptographic signature verification
 6. Data is visualized using Recharts components
-

6. Features

Implemented Features

1. **Lead Capture & Storage:** Email/password and social login with Firebase Authentication
2. **Portfolio Tracking:** Real-time portfolio value monitoring with asset breakdown
3. **Analytics Dashboard:** Performance charts, asset allocation visualization
4. **User Analysis:** Community portfolio insights and comparison
5. **Theme System:** Dark/light mode with system preference detection
6. **Wallet Authentication:** Crypto wallet connection and verification

Feature-Subject Mapping

- **DSA:** Hashing (signature verification), Graphs (portfolio relationships)
 - **NoSQL:** Firebase Firestore collections (users, portfolios)
 - **Business 2:** Customer acquisition funnel, loyalty tracking through engagement metrics
-

7. Algorithms & Data Structures

Implemented Algorithms

- **Hashing:** Wallet signature verification
- **Graphs:** Portfolio visualization (Recharts)
- **Sorting:** User portfolio ranking

Usage & Complexity

- **Hashing (Signature Verification):** $O(1)$
 - **Graph Algorithms (Recharts):** $O(n)$
 - **Sorting (User Ranking):** $O(n \log n)$
-

8. Database Design

Firestore Collections

Users Collection

```
{  
  "uid": "string",  
  "email": "string",  
  "name": "string",  
  "walletAddress": "string",  
  "photoURL": "string",  
  "createdAt": "timestamp",  
  "lastLogin": "timestamp",  
  "preferences": {  
    "currency": "string",  
    "theme": "string",
```

```
    "notifications": "boolean"
  }
}
```

Portfolios Collection

```
{
  "uid": "string",
  "totalValue": "number",
  "totalInvested": "number",
  "holdings": [
    {
      "symbol": "string",
      "name": "string",
      "amount": "number",
      "purchasePrice": "number",
      "currentPrice": "number"
    }
  ],
  "transactions": [
    {
      "id": "string",
      "type": "string",
      "asset": "string",
      "amount": "number",
      "price": "number",
      "timestamp": "string"
    }
  ]
}
```

9. API & Backend Logic

Endpoints

1. **POST** `/api/auth/wallet` - Wallet authentication and user creation
2. **GET** `/api/crypto-data` - Fetch cryptocurrency market data
3. **GET** `/api/portfolio` - Retrieve portfolio data (mock)
4. **POST** `/api/users` - Create user (mock)
5. **GET** `/api/users/list` - List all users with portfolio data
6. **GET** `/api/users/[uid]` - Get specific user data

CRUD Operations

- **Create:** User registration, portfolio creation
 - **Read:** Portfolio data, user information, market data
 - **Update:** User profile, portfolio updates
 - **Delete:** Not implemented
-

10. Frontend Structure

Main Pages

1. Landing Page
2. Login / Signup
3. Dashboard
4. User Analysis

5. Crypto Data

UI Components

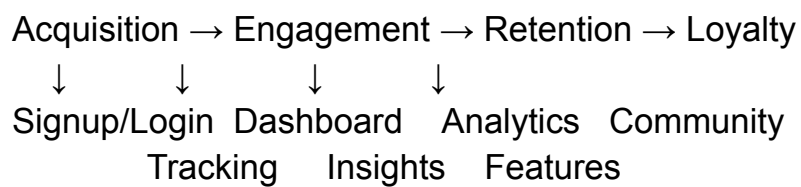
- CryptoPieChart
- CryptoLineChart
- ResponsiveLayout
- ThemeToggle

Visualization

- Recharts for charts
 - Custom SVGs for UI elements
-

11. Business Logic

Customer Journey Flow



Lead Qualification

- Email verification during signup
- Wallet connection as premium indicator

- Portfolio value as engagement metric

Conversion Tracking

- Login frequency
- Portfolio update activity
- Alert creation and interaction

Loyalty Metrics

- User score based on:
 - Portfolio ROI (40%)
 - Diversification (25%)
 - Account age (20%)
 - Recent activity (15%)
-

12. Operations Module

User Management

- Profile storage & retrieval
- Portfolio data management
- Authentication state handling

Data Operations

- Real-time portfolio value calculation
- Historical performance tracking
- Asset allocation analysis

System Operations

- Theme preference management
 - Notification settings
 - Currency conversion preferences
-

13. Testing

Test Files

1. `functions/__tests__/basic.test.ts`
2. `functions/__tests__/simple.test.ts`

Approach

- Unit testing
- Component testing
- Integration testing
- Manual testing for flows

Coverage

- Authentication flows (partial)
 - API endpoints (partial)
 - UI components (limited)
 - Business logic (minimal)
-

14. Results / Outputs

Computed KPIs

- User Score
- Portfolio ROI
- Asset Allocation
- Diversification Index
- Engagement Metrics

Visual Outputs

- Portfolio performance charts
 - Asset allocation pie charts
 - User comparison dashboards
 - Community insights visualizations
-

15. Conclusion

Technical Achievements

- Multi-method authentication
- Responsive UI with dark/light theme
- Real-time Recharts visualizations
- Wallet cryptographic verification
- Firebase integration

Business Achievements

- Customer acquisition pathway implemented
- Engagement metrics captured
- Community comparison dashboard
- Data-driven user insights

Screenshots to Add:

- Login/Signup pages
- Dashboard overview
- Wallet authentication flow

16. References

- Next.js, React
 - Firebase Auth & Firestore
 - Recharts
 - Tailwind CSS
 - Ethers.js & Viem
 - CoinMarketCap API
 - Reown AppKit
-

17. Code Snippets

Wallet Authentication

```
const isValid = await verifyMessage({
  address: address as `0x${string}`,
  message,
  signature: signature as `0x${string}`,
});
```

User Score Calculation

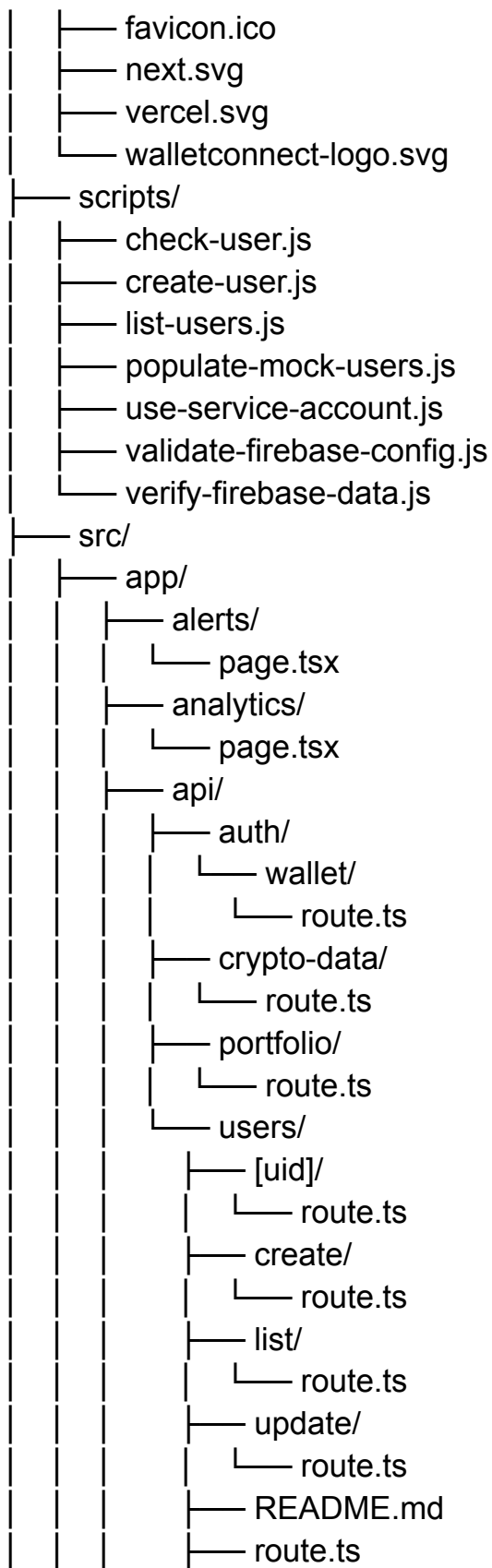
```
const score = (
  (roi * 0.4) +
  (diversificationScore * 0.25) +
  (experienceScore * 0.2) +
  (activityScore * 0.15)
);
```

Theme Context

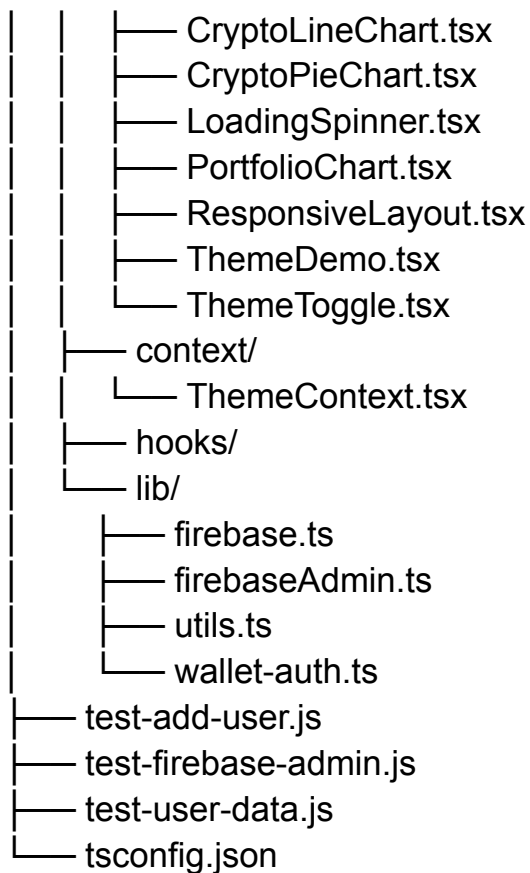
```
useEffect(() => {  
  document.documentElement.classList.remove("light", "dark");  
  document.documentElement.classList.add(theme);  
  localStorage.setItem("theme", theme);  
}, [theme]);
```

18. Repository Structure

```
crpto-app/  
├── README.md  
├── THEME_SYSTEM.md  
├── businessmodel_CryptoTracker.xlsx  
├── components.json  
├── firebase-service-account-template.json  
├── firebase.json  
├── firestore.rules  
├── functions/  
│   ├── __tests__/  
│   │   ├── basic.test.ts  
│   │   ├── simple.test.ts  
│   │   └── tsconfig.json  
│   ├── lib/  
│   │   └── index.js  
│   ├── src/  
│   │   └── index.ts  
│   ├── README.md  
│   ├── package-lock.json  
│   ├── package.json  
│   └── tsconfig.json  
├── mock-users.json  
├── next.config.js  
├── next.config.ts  
├── package-lock.json  
├── package.json  
├── postcss.config.mjs  
└── public/
```

```
|
|
|   └─ test-user-creation.ts
|
| └─ crypto-data/
|   └─ page.tsx
|
| └─ dashboard/
|   └─ page.tsx
|
| └─ demo/
|   └─ page.tsx
|
| └─ help/
|   └─ page.tsx
|
| └─ login/
|   └─ page.tsx
|
| └─ portfolio/
|   └─ page-firestore.tsx
|   └─ page.tsx
|
| └─ profile/
|   └─ page.tsx
|
| └─ settings/
|   └─ page.tsx
|
| └─ signup/
|   └─ page.tsx
|
| └─ test-charts/
|   └─ page.tsx
|
| └─ test-crypto/
|   └─ page.tsx
|
| └─ test-dock/
|   └─ page.tsx
|
| └─ test-links/
|   └─ page.tsx
|
| └─ users/
|   └─ page.tsx
|
| └─ globals.css
|
| └─ layout.tsx
|
| └─ page.tsx
|
| └─ components/
|   └─ ui/
|     └─ dock.tsx
|
| └─ CryptoBarChart.tsx
```



19. Sprint 10-Day Plan

Day 1-2: Requirements & architecture review

Day 3-4: Authentication enhancement

Day 5-6: Dashboard expansion

Day 7-8: Analytics implementation

Day 9: Testing & bug fixes

Day 10: Documentation & deployment

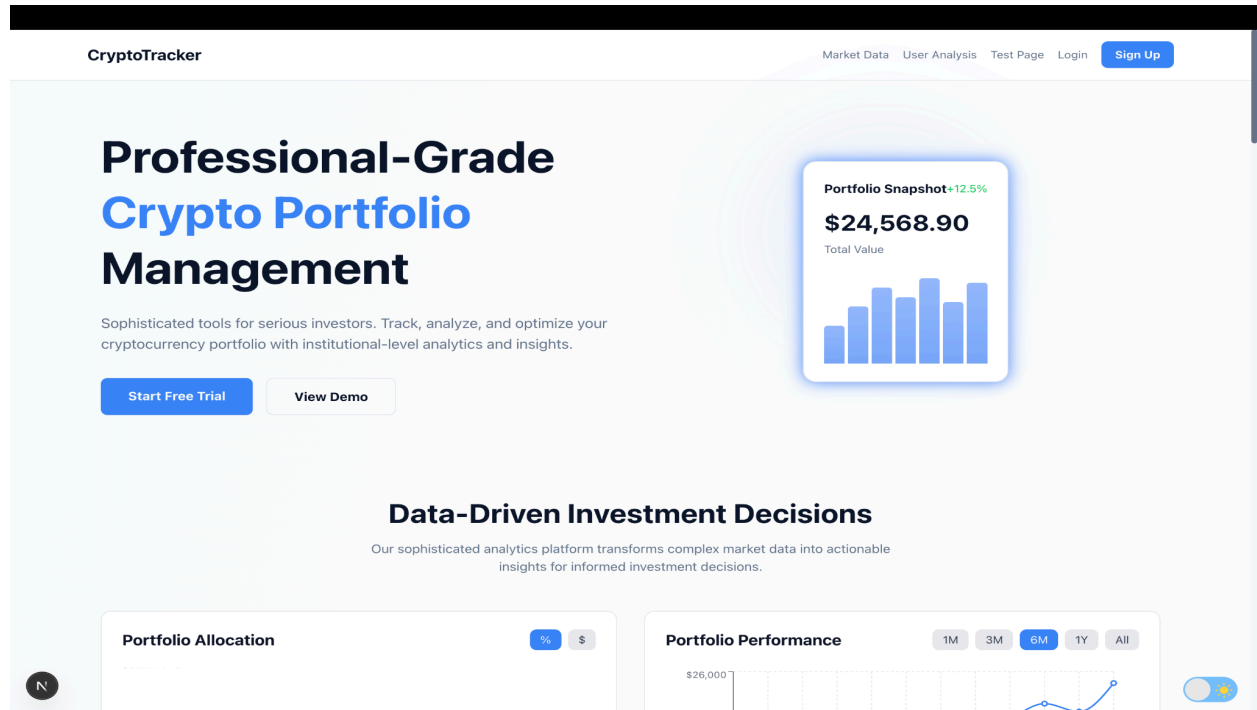
20. Demo Script

1. Landing page introduction

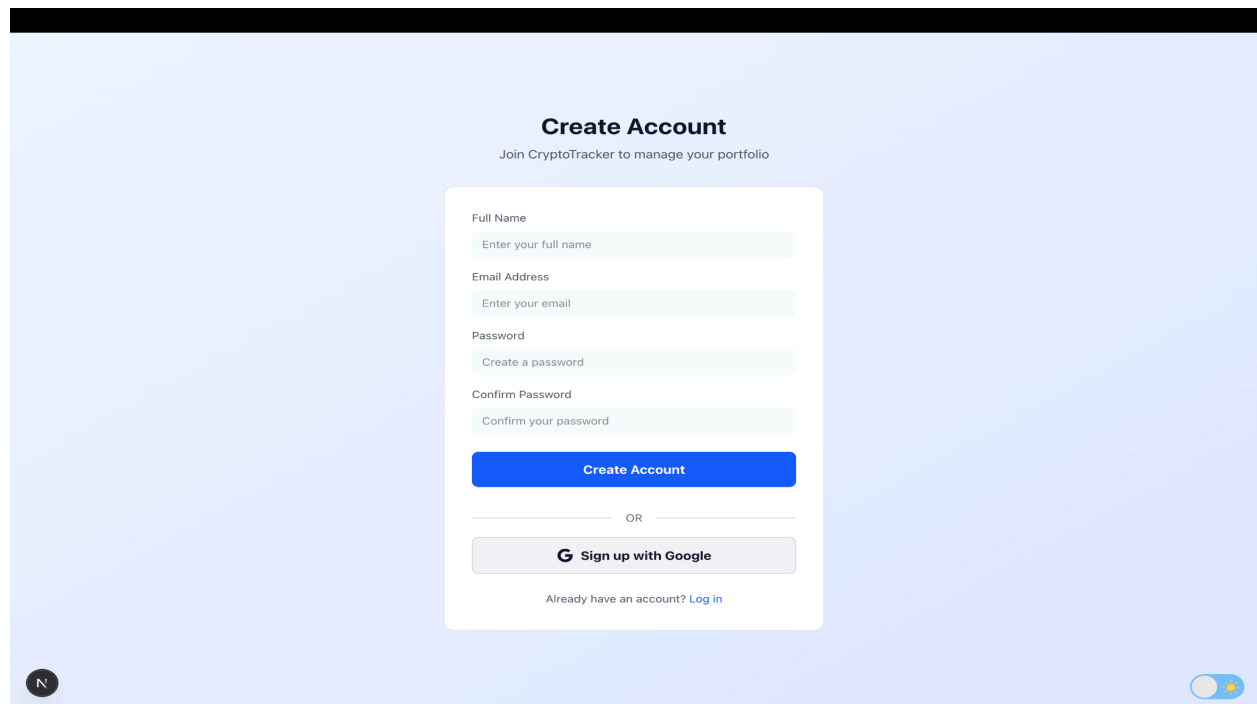
2. Signup/Login demonstration
3. Dashboard walkthrough
4. Portfolio analysis features
5. User comparison functionality
6. Theme switching demonstration
7. Wallet authentication flow
8. Q&A session

Screenshots:

A. Landing Page — Lead capture form



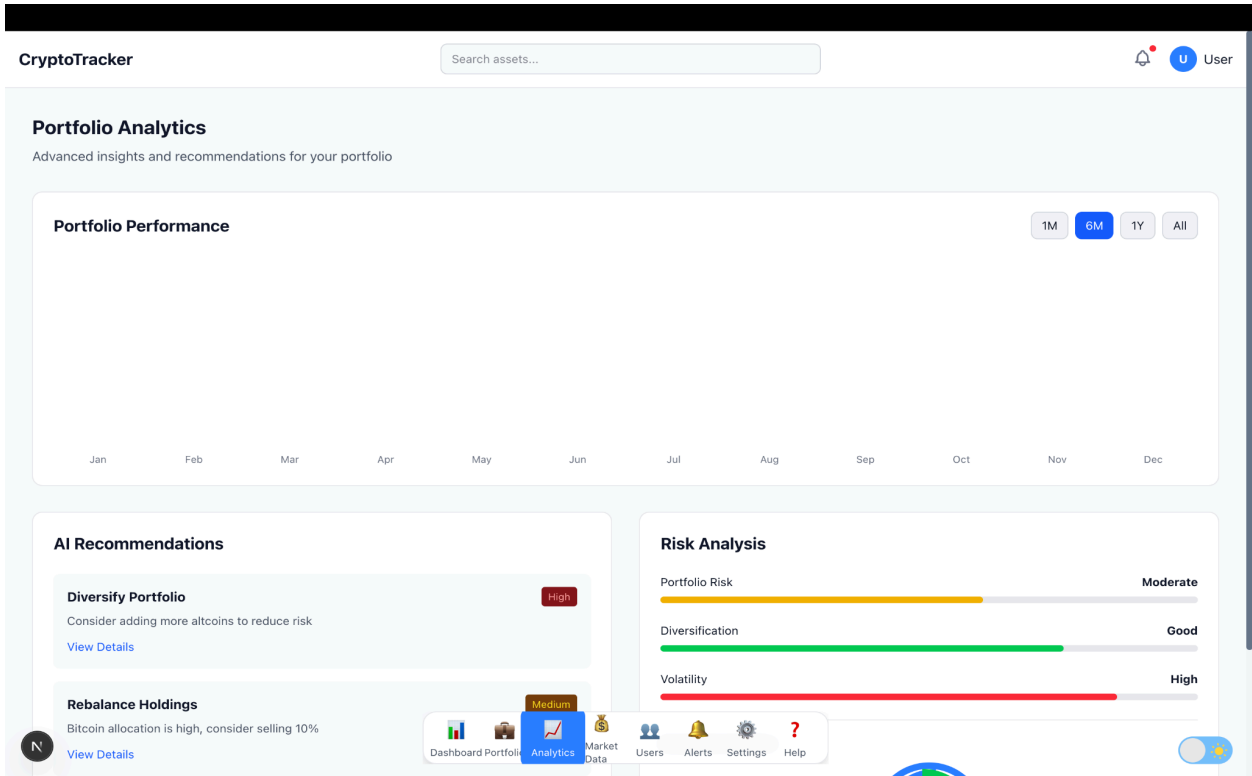
B. Signup / Authentication Page



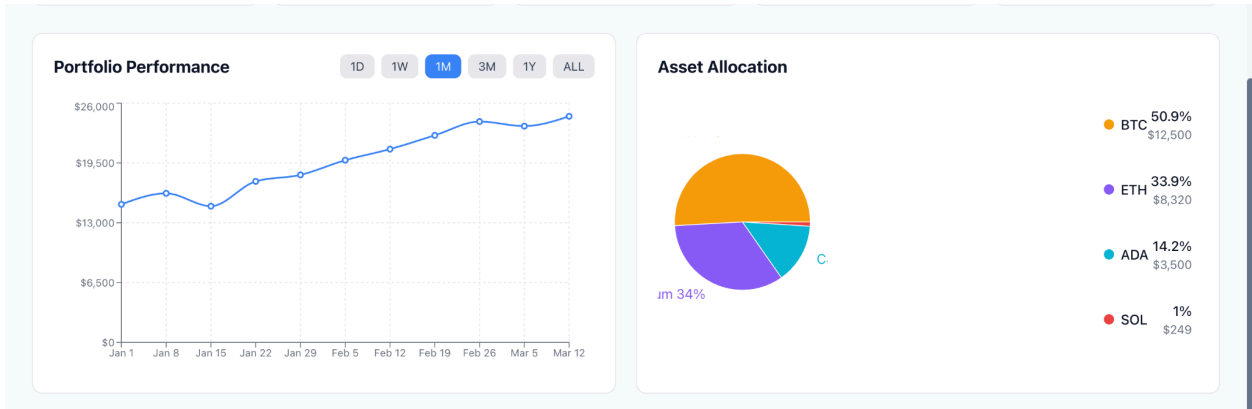
C. Demo Import Wallet / WalletConnect



E. RFM Page (Analytics)



F. Referral Graph Analysis (Graph BFS)



K. Code Repository Screenshot

